



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE FOUR: JAVA CONTROL STRUCTURES

UNIT ONE: CONDITIONAL STATEMENTS AND DECISION MAKING

CONDITIONAL STATEMENTS AND DECISION MAKING

Learning Objectives

By the end of this unit, you will be able to:

- **define control structures and explain their purpose in programming.**
- **identify and describe different types of control structures.**
- **use conditional statements (if, if-else, if-else-if, switch, and the ternary operator) to manage program flow.**
- **evaluate Boolean expressions for use in decision-making.**
- **develop Java programs that use conditional logic to solve practical problem**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the concept of control structures.**
- **identify and explain types of control structure.**
- **apply conditional statements (if, if-else, if-else-if, switch, and the ternary operator) to control program flow based on specific conditions.**
- **understand the role of Boolean expressions in decision-making.**
- **write Java programs that incorporate conditional logic to solve real-world problems.**

Introduction

- **Control structures are essential tools that guide the flow of a program. They enable decision-making, repetition, and error handling. This unit focuses on decision-making structures, which allow a program to respond to different inputs or conditions by choosing which code to execute.**
- **Instead of running code in sequentially, these structures let the program test conditions and decide the code block to run. Java offers several options for this, including if, if-else, if-else-if, switch, and the ternary (?:) operator, helping developers manage program flow effectively.**

What are Control Structures?

- **Control structures in programming are special instructions that manage the flow of execution in a program. They allow the program to make decisions, repeat actions, or choose between different paths based on certain conditions.**
- **Control structures are generally categorized into three groups:**
 1. **Conditional (Decision-Making),**
 2. **Loop (Repetition), and**
 3. **Branching (Jump/Control Transfer).**

Conditional Structures

These execute specific blocks of code only when certain conditions are met. Common examples include:

- 1. if statement**
- 2. if-else statement**
- 3. else-if ladder**
- 4. switch statement**

The if Statement

This evaluates a Boolean expression and executes a block of code only if the condition is true. The syntax is:

```
if (condition) {  
    // Code to be executed if condition is true  
}
```

```
public class IfStatement {  
    public static void main(String[] args) {  
        int age = 19;  
        if (age >= 18) {  
            System.out.println("You are eligible to vote!");  
        }  
    }  
}
```

The if Statement (2)

The code above demonstrates a conditional statement that only executes if the condition is true.

The output of the code is:

“You are eligible to vote!”

The code is skipped if the condition is false.

The if-else Statement

Executes one block if the condition is true, another block if false. The syntax is:

```
if (condition) {  
    //Execute code if condition is true  
} else {  
    // Executes code if condition is false  
}
```

The if-else Statement (2)

```
public class IfElseStatement {  
    public static void main(String[] args) {  
        int num = 10;  
        if (num % 2 == 0) {  
            System.out.println(num + " is even");  
        } else {  
            System.out.println(num + " is odd");  
        }  
    }  
}
```

The code checks whether a given integer (number) is even or odd using a simple if-else conditional statement. The output of the code in the Figure above is:

10 is even

The else-if Ladder

This is used for multiple conditions. The program checks each condition in sequence until one is true. The syntax is:

```
if (condition1) {  
    // Executes if condition1 is true  
} else if (condition2) {  
    // Executes if condition1 is false and condition2 is true  
} else if (condition3) {  
    // Executes if conditions 1 and 2 are false and  
    // condition3 is true  
} else {  
    // Executes if none of the above conditions are true  
}
```

The else-if Ladder (2)

```
public class IfElseStatementTwo {  
    public static void main(String[] args) {  
        int score = 75;  
  
        if (score >= 90) {  
            System.out.println("Your grade is A");  
        } else if (score >= 80) {  
            System.out.println("Your grade is B");  
        } else if (score >= 70) {  
            System.out.println("Your grade is C");  
        } else if (score >= 60) {  
            System.out.println("Your grade is D");  
        } else {  
            System.out.println("Your grade is F");  
        }  
    }  
}
```

The output from the code snippet is:

Your grade is C

The Switch Statement

- The switch statement streamlines multiple if-else checks by evaluating an expression once and executing the block of code that matches the corresponding case value.
- The break statement is used to terminate a specific case, immediately jumping to the end of the switch block.
- The syntax is:

```
switch (expression) {  
    case value 1:  
        // code block  
        break;  
    case value 2:  
        // code block  
        break;  
    default:  
        // code block  
}
```

The Switch Statement

- Without a break, the program continues executing the subsequent case(s), a behavior known as "fall-through," until it encounters a break or reaches the end of the switch.

The Switch Statement: Example 1

- The code in the Figure uses a switch statement to display a message based on the value of the category variable.
- It checks the value of category and matches it to one of the defined case blocks.

```
public class Switch1 {  
    public static void main(String[] args) {  
        int category = 1;  
        switch (category) {  
            case 1:  
                System.out.println("Pre-teen");  
                break;  
            case 2:  
                System.out.println("Teenager");  
                break;  
            case 3:  
                System.out.println("Adult");  
                break;  
            default:  
                System.out.println("Senior Adult!");  
                break;  
        }  
    }  
}
```

The Switch Statement: Example 2

```
public class SwitchStringExample {  
    public static void main(String[] args) {  
        String level = "intermediate";  
        switch (level) {  
            case "beginner":  
                System.out.println("Welcome, beginner!");  
                break;  
            case "intermediate":  
                System.out.println("You're doing well!");  
                break;  
            case "advanced":  
                System.out.println("You're almost an expert!");  
                break;  
            default:  
                System.out.println("Unknown level.");  
        }  
    }  
}
```

Ternary Operator

- The ternary operator is a compact way of writing an if-else statement. It is often used for simple conditional expressions where you want to assign a value or return a result based on a condition. The syntax is:

dataType variable = (**condition**) ? val_if_true : val_if_false;

- The code below uses ternary operator to find and print the maximum of two integers.

```
public class MaxNumbers {  
    public static void main(String[] args) {  
        int a = 10, b = 20;  
        int max = (a > b) ? a : b;  
        System.out.println("Maximum is: " + max);  
    }  
}
```

Ternary Operator: Example 2

```
public class TemperatureCheck {  
    public static void main(String[] args) {  
        int temp = 25;  
        System.out.println(temp > 30 ? "It's hot" : "It's cool");  
    }  
}
```

The code above:

- Declares a variable, *temp* and initializes it to 25.
- Use a ternary operator to check:
 - If *temp* >= 30, prints "*It's hot*"
 - Otherwise, prints "*It's cool*"
- The ternary operator is used inside the println statement

Ternary Operator: Example 3

```
public class GradeCalculator {  
    public static void main(String[] args) {  
        int score = 55;  
        String grade = (score >= 90) ? "A" :  
            (score >= 80) ? "B" :  
            (score >= 70) ? "C" : "F";  
  
        System.out.println("Grade: " + grade);  
    }  
}
```

- This code assigns a letter grade based on the score using ***nested ternary operators*** and prints the result.
- When you run the code, the output is:

Grade: F

Nested if Statement

Nested if statements occur when an if or else block contains another if or else statement inside it. This allows you to test multiple conditions in a hierarchical or sequential manner. It is used to check complex conditions that depend on multiple criteria and to perform different actions based on combinations of conditions. The syntax is:

```
if (condition1) {  
    if (condition2) {  
        // Executes if condition1 and condition2 are true  
    } else {  
        // Executes if condition1 is true and condition 2 is false  
    }  
} else {  
    // Executes if condition1 is false  
}
```

Nested if Statement (2)

The code below checks if a number is positive and then determines if it's even or odd, printing the result accordingly.

```
public class NumberCheck {  
    public static void main(String[] args) {  
        int num = 10;  
  
        if (num > 0) {  
            if (num % 2 == 0) {  
                System.out.println("Positive even number");  
            } else {  
                System.out.println("Positive odd number");  
            }  
        } else {  
            System.out.println("Non-positive number");  
        }  
    }  
}
```

Nested if Statement (3)

The code is used to checks if a person is 18 or older and has an ID; if both conditions are true, it prints "*Eligible to vote*".

```
public class VotingEligibility {  
    public static void main(String[] args) {  
        int age = 20;  
        boolean hasID = true;  
        if (age >= 18) {  
            if (hasID) {  
                System.out.println("Eligible to vote");  
            } else {  
                System.out.println("ID required to vote");  
            }  
        } else {  
            System.out.println("Not eligible to vote");  
        }  
    }  
}
```

The output of the code is: *Eligible to vote*

Nested if Statement (4)

```
public class LeapYearCheck {  
    public static void main(String[] args) {  
        int year = 2024;  
  
        if (year % 4 == 0) {  
            if (year % 100 == 0) {  
                if (year % 400 == 0) {  
                    System.out.println(year + " is a leap year");  
                } else {  
                    System.out.println(year + " is not a leap year");  
                }  
            } else {  
                System.out.println(year + " is a leap year");  
            }  
        } else {  
            System.out.println(year + " is not a leap year");  
        }  
    }  
}
```

Nested if Statement (4)

In the Figure above, the code (titled Leap Year Checker) checks if a given year is a leap year based on the rules:

- divisible by 4
- not divisible by 100 unless also divisible by 400.

It prints the result accordingly.

The code will output:

2024 is a leap year

In-Text Questions (ITQs)

- 1. What are control structures in Java, and why are they important?**
- 2. How does the if-else statement work?**
- 3. What is the purpose of the switch statement, and how does it differ from an if-else ladder?**

In-Text Answers (ITAs)

- 1. Control structures are programming constructs that manage the flow of execution in a program. They are important because they allow the program to make decisions, repeat actions, and control which parts of the code run based on certain conditions.**
- 2. The if-else statement executes one block of code if a condition is true, and another block if the condition is false.**
- 3. The switch statement evaluates an expression once and executes the matching case block, which makes it more efficient for multiple fixed values. Switch uses the break statements to avoid fall-through.**

Conclusion

- **Understanding and applying decision-making structures is essential in programming. Java provides versatile conditional constructs (if, switch, ternary, etc.) that enable developers to control program flow based on logic and input.**
- **By mastering these tools, programmers can build responsive, flexible, and intelligent applications capable of handling a wide range of real-life scenarios.**

Summary

This unit covers control structures used for decision-making in Java. These statements let programs choose different actions based on conditions. Key types include:

- **if: runs code if a condition is true**
- **if-else: selects between two options**
- **else-if ladder: checks multiple conditions**
- **switch: handles several fixed values efficiently**
- **ternary operator: a short form of if-else**
- **nested if: checks conditions within conditions**